



Research article

NAS-Optimized tinyML intrusion detection for ultralow-power IoT edge devices

Iván Ortiz-Garcés ^a, Milton Román-Cañizares ^b, Pablo Palacios ^c,
William Villegas-Ch ^{a,*}

^a Escuela de Ingeniería en Ciberseguridad, FICA, Universidad de Las Américas, Av. de los Granados y Colimes, Quito, 170125, Pichincha, Ecuador

^b Facultad de Hábitat, Infraestructura y Creatividad, Pontificia Universidad Católica del Ecuador, Quito, 170525, Pichincha, Ecuador

^c Escuela de Informática y Telecomunicaciones, Universidad Diego Portales, Santiago, Región Metropolitana, 8370191, Chile

ARTICLE INFO

Keywords:

TinyML intrusion detection
Neural architecture search (NAS)
IoT Edge security
Multidomain dataset alignment

ABSTRACT

Intrusion detection in Internet of Things (IoT) environments faces a persistent gap between the high predictive performance of deep learning models and their feasibility on ultra-low-power embedded devices, as many existing approaches exceed the memory, latency, and energy constraints of edge microcontrollers and rely on domain-specific datasets that limit cross-domain generalization. This work proposes a Tiny Machine Learning (TinyML) intrusion detection framework optimized through Neural Architecture Search (NAS) and trained on a multidomain, feature-aligned dataset integrating heterogeneous sources, including TON_IoT, IoT-23, Edge-IIoTset, and a controlled experimental dataset, using statistical alignment and dimensionality reduction to obtain compact representations of 32-64 features while preserving over 99% of the informative variance. Experimental results achieve F1-macro scores above 0.95 on IoT-23 and 0.98 in controlled scenarios. At the same time, hardware-level validation confirms deployment feasibility with an inference latency of 2.48 ms, RAM usage below 20 KB, and energy consumption of 0.39 mJ per inference, demonstrating the viability of efficient and autonomous intrusion detection directly on IoT edge devices.

1. Introduction

The rapid expansion of the Internet of Things (IoT) has led to highly distributed infrastructures composed of heterogeneous, resource-constrained devices operating in dynamic and often critical environments [1]. These systems are increasingly exposed to diverse cyber threats, including denial-of-service attacks, automated scanning, and protocol-level exploitation, making timely and reliable intrusion detection at the device level a key operational requirement [2]. In edge scenarios with limited connectivity, strict latency constraints, and privacy restrictions, centralized security mechanisms are often insufficient.

Deep learning-based intrusion detection systems (IDS) have demonstrated strong performance in controlled settings; however, their deployment on IoT edge devices remains limited. Many approaches rely on computationally intensive architectures with high memory and energy requirements, making them unsuitable for ultra-low-power microcontrollers [3]. Even when post-training optimizations such as pruning or quantization are applied, these models frequently fail to meet strict hardware constraints, particularly in devices with memory budgets below 128 KB [4]. This gap between detection capability and deployment feasibility remains a central limitation.

* Corresponding author.

E-mail addresses: ivan.ortiz@udla.edu.ec (I. Ortiz-Garcés), mnroman@puce.edu.ec (M. Román-Cañizares), pablo.palacios@mail.udpl.cl (P. Palacios), william.villegas@udla.edu.ec (W. Villegas-Ch).

<https://doi.org/10.1016/j.iot.2026.102034>

Received 21 December 2025; Received in revised form 24 June 2026; Accepted 13 July 2026

Available online 16 July 2026

2542-6605/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Tiny Machine Learning (TinyML) offers a promising alternative by enabling on-device inference under strict resource constraints [4]. Nevertheless, designing effective TinyML-based IDS models remains challenging due to non-stationary traffic patterns, class imbalance, and restricted representational capacity. Neural Architecture Search (NAS) offers a systematic mechanism to explore compact architectures while integrating constraints on latency, memory, and energy consumption, making it particularly suitable for embedded deployment scenarios [2].

In addition to architectural constraints, dataset heterogeneity introduces further complexity. Widely used datasets such as TON_IoT, IoT-23, and Edge-IIoTset exhibit substantial differences in feature distributions, class balance, and statistical properties, which complicates cross-domain generalization and reproducibility [3]. Without explicit alignment mechanisms, these inconsistencies can lead to biased evaluations and misleading performance estimates [5].

Recent lightweight and edge-oriented intrusion detection approaches, such as hybrid multimodal frameworks designed for edge-enabled IoT devices [6], have demonstrated improved efficiency and adaptability in IoT environments with limited computational resources. However, these methods primarily focus on model-level optimization or modality integration, without explicitly incorporating hardware-aware architecture search, multidomain data alignment, and deployment validation on microcontrollers. This limitation restricts their applicability in scenarios where both generalization and strict resource constraints must be addressed simultaneously.

This work proposes a TinyML-based intrusion detection framework that integrates multidomain feature-space alignment with hardware-aware neural architecture search under strict resource constraints. The approach constructs a hybrid dataset through controlled dimensionality reduction, enabling compact feature representations (32–64 dimensions) suitable for embedded deployment.

The framework integrates heterogeneous datasets (TON_IoT, IoT-23, Edge-IIoTset, and a controlled experimental dataset) and evaluates detection capability and deployment feasibility under multidomain conditions. The optimized model achieves F1-macro values above 0.95 on IoT-23 and 0.98 on the controlled dataset while maintaining low computational overhead.

Hardware-level validation on a Cortex-M class microcontroller confirms the feasibility of the approach, with inference latency below 2.5 ms, RAM usage under 20 KB, and energy consumption of approximately 0.39 mJ per inference. These results demonstrate that incorporating hardware constraints directly into the architecture search process enables practical on-device intrusion detection in resource-constrained IoT environments.

2. Literature review

TinyML has emerged as a key paradigm for enabling machine learning inference on resource-constrained IoT devices. Foundational studies such as Dutta and Bharali [7] and Schizas et al. [4] demonstrate the feasibility of deploying models on microcontrollers using frameworks such as TensorFlow Lite Micro, but also highlight that model design remains largely manual and dependent on expert knowledge, limiting scalability.

Subsequent work identifies four dominant trends in TinyML systems: energy optimization, structural reduction through pruning and quantization, expansion of edge applications, and the gradual incorporation of automated design mechanisms such as Neural Architecture Search (NAS) [2,8]. Despite these advances, the lack of integrated hardware-aware design strategies remains a major limitation, particularly in ultra-constrained environments where memory and energy budgets are critical.

In the cybersecurity domain, most intrusion detection systems (IDS) rely on complex deep learning architectures, including CNNs, RNNs, and autoencoders, which achieve high detection performance but are unsuitable for embedded deployment due to their computational cost [3,9]. Recent approaches such as EvoAAE, MoARNN-AM, and MODEO-CNN have improved detection accuracy by incorporating advanced architectures and optimization techniques; however, these methods typically prioritize predictive performance and do not explicitly address hardware constraints, limiting their applicability in TinyML-based IoT environments.

To bridge this gap, several studies have explored lightweight IDS solutions. For instance, SMEESI [10] integrates compact architectures with dynamic power management, achieving reduced energy consumption while maintaining competitive accuracy. Similarly, GraBoost-AAGT [11] employs bio-inspired optimization for hyperparameter tuning. While these approaches improve efficiency, they rely on predefined or manually adjusted architectures, which restrict adaptability to different hardware platforms.

Comparative evaluations such as Hamdouchi and Idri [12] and Sharma [13] demonstrate that lightweight models can outperform more complex architectures when deployed on real microcontrollers, particularly in terms of latency and energy consumption. However, these studies also reveal that the architectural search space remains limited and does not leverage automated optimization strategies.

Specialized studies targeting specific threats, such as Mirai botnet detection [14], further confirm the feasibility of TinyML-based IDS in next-generation IoT environments. Nevertheless, these works predominantly rely on conventional classifiers and do not incorporate automated architecture selection or multi-objective optimization.

Local inference enabled by TinyML reduces data exposure and enhances system resilience [5]. However, recent findings also highlight new vulnerabilities, including model extraction attacks and deployment inconsistencies [15,16]. At the same time, large-scale models designed for smart environments offer advanced analytical capabilities but remain impractical for resource-constrained IoT devices [17].

The literature reveals three critical limitations: (1) the predominance of manually designed or fixed architectures without systematic optimization; (2) limited evaluation across heterogeneous datasets, restricting generalizability; and (3) the absence of integrated multi-objective optimization frameworks that simultaneously consider detection accuracy, memory usage, energy consumption, and latency. These gaps motivate the development of TinyML-based intrusion detection systems optimized via NAS and evaluated under realistic multidomain conditions.

3. Materials and methods

3.1. Hybrid dataset engineering and multidomain correlation

The proposed system is trained and validated using a hybrid dataset constructed from TON_IoT, IoT-23, Edge-IIoTset, and a complementary controlled evaluation environment [18–20]. These sources capture heterogeneous IoT scenarios, including domestic environments, industrial processes, multisensor telemetry, and network-level traffic, thereby enabling the definition of a unified training space that reflects real-world variations in protocols, topologies, sampling rates, and operating conditions [1].

TON_IoT provides multimodal data combining environmental sensors, network flows, and system logs, with heterogeneous sampling rates ranging from low-frequency environmental measurements to higher-frequency inertial signals [21]. IoT-23 focuses on packet-capture-based flow traffic from commercial devices, including malware activity such as Mirai and Torii, and requires specialized preprocessing due to its non-uniform temporal structure [22]. Edge-IIoTset contributes to industrial telemetry with higher temporal regularity and process-oriented attack patterns that affect physical variables such as temperature and pressure, requiring alignment mechanisms to ensure compatibility with flow-based and sensor-based data [19].

In addition to public datasets, a controlled evaluation environment is incorporated to extend operational coverage beyond existing repositories. This environment enables observation of underrepresented conditions, including constrained traffic patterns, microcontroller-level processing limitations, and gradual degradation. Rather than generating fully artificial samples, this component introduces controlled, perturbation-based variations informed by empirical observations, preserving the statistical structure of the original data while simulating realistic variability. The integration of these sources introduces controlled heterogeneity, which is essential for training TinyML models that maintain stability and generalization under strict resource constraints. To ensure semantic compatibility across datasets, labels are harmonized into a common representation based on coarse-grained categories, avoiding direct alignment of dataset-specific attack taxonomies.

3.1.1. Temporal alignment, normalization and feature-space projection

Given the heterogeneity in temporal resolution and data structure, all datasets are aligned to a common time scale via periodic resampling over a 50–100 ms range. This interval is selected to preserve relevant temporal dynamics while remaining compatible with the processing capabilities of low-power microcontrollers. The selected resampling interval represents a trade-off between preserving relevant temporal patterns and ensuring computational feasibility on resource-constrained devices, with a focus on aggregated behavioral dynamics rather than fine-grained temporal signatures.

Feature normalization is applied according to the nature of each dataset. Flow-based variables from IoT-23 are transformed using logarithmic scaling to mitigate burst effects. At the same time, sensor measurements from TON_IoT and Edge-IIoTset are standardized using Z-score normalization to compensate for magnitude differences across modalities. Categorical log data is encoded using low-dimensional hashing to preserve structural relationships without increasing memory requirements.

To ensure compatibility with TinyML constraints, a two-stage feature reduction process is applied. First, features are selected using mutual information criteria, retaining only the most informative variables with respect to the target labels:

$$S = \{x_i \mid I(x_i; y) > \tau\} \quad (1)$$

This step reduces the original feature space (typically exceeding 200 dimensions) to a subset of approximately 80–120 features.

In the second stage, dimensionality reduction is applied to obtain a compact representation of 32–64 features, depending on the available memory budget of the target device. Linear projections are applied in domains exhibiting high feature correlation and low intrinsic nonlinearity, enabling efficient dimensionality reduction with minimal information loss [23]. In contrast, lightweight autoencoders with fewer than 5K parameters are used when the data presents nonlinear dependencies and coupled feature interactions, where linear transformations are insufficient to preserve discriminative patterns. The resulting representation preserves the most relevant structural information while enabling efficient deployment on embedded hardware.

3.1.2. Controlled evaluation environment and domain-bias mitigation

The integration of heterogeneous datasets introduces the risk of domain bias, particularly when certain datasets dominate in size or variability. To mitigate this issue, a domain-aware training strategy is implemented that combines stratified balancing and weighted optimization.

The overall training objective is defined as a weighted aggregation of domain-specific losses:

$$\mathcal{L} = \sum_d \alpha_d \mathcal{L}_d, \quad \text{with} \quad \alpha_d = \frac{1}{N_d} \quad (2)$$

where $\mathcal{L}_d$ represents the loss associated with domain d , and N_d is the number of samples in that domain. This formulation prevents larger datasets from dominating the optimization process, promoting balanced learning across domains.

To further reduce bias, reinforcement of underrepresented scenarios is performed in the latent feature space rather than in the raw input domain. This strategy preserves the physical plausibility of the original data while enabling the model to explore low-density regions of the representation space.

The controlled evaluation environment complements this process by introducing empirically grounded variations that reflect realistic operational conditions, including temporal fluctuations, constrained traffic loads, and gradual degradation patterns. These variations are derived from observed statistical distributions and remain within realistic bounds, ensuring that the resulting training data remains consistent with real-world IoT behavior. By integrating multidomain data alignment, controlled variability, and

domain-aware optimization, the proposed framework establishes a robust and hardware-compatible training pipeline for TinyML-based intrusion detection systems.

3.2. Feature compression and hardware-Constrained preprocessing

The feature engineering process is designed to ensure that the final representation satisfies the physical constraints of the target microcontroller while preserving sufficient discriminative capacity for intrusion detection. The resulting pipeline operates within typical TinyML limits, including 24-96 KB of RAM, 192-512 KB of FLASH memory, and inference latencies below 25 ms, consistent with ARM Cortex-M and ESP32-class devices.

The initial feature space derived from IoT-23, TON_IoT, Edge-IIoTset, and the controlled evaluation environment comprises more than 200 variables. To ensure feasibility on embedded hardware, a reduction stage is applied based on mutual information and model-agnostic relevance criteria. Features are selected according to their mutual information with respect to the target labels:

$$S = \{x_i \mid I(x_i; y) > \tau\} \quad (3)$$

This process reduces the feature space to approximately 80–120 variables. To complement this selection, SHAP-based analysis is performed using a lightweight auxiliary classifier, enabling the identification of nonlinear interactions without introducing bias toward the final deployed model.

Given the use of quantization-aware training (QAT), feature preprocessing ensures compatibility with 8-bit integer representations. Continuous variables are scaled to bounded ranges, while heavy-tailed distributions, such as packet rates and inter-event delays, are stabilized using logarithmic transformations to reduce variance and prevent saturation effects during quantization. Categorical variables are encoded using compact binary or low-cardinality integer representations, avoiding the dimensional expansion associated with one-hot encoding. Feature distributions are subsequently standardized and rescaled to maintain compatibility with symmetric quantization ranges.

The reduced feature set is projected into a compact latent representation with 32–64 dimensions, depending on the memory constraints of the target device. This projection is implemented using either linear transformations or ultracompact autoencoders with fewer than 5K parameters. In domains with predominantly nonlinear relationships, lightweight autoencoders preserve structural dependencies while maintaining a minimal parameter footprint.

3.3. Hardware-Aware neural architecture search (NAS)

The NAS process constitutes the core of the proposed system, as it determines the final TinyML architecture under strict hardware constraints. Unlike conventional NAS approaches, the search is explicitly tailored to ultra-low-power microcontrollers, ensuring that each candidate model is evaluated for both detection performance and deployment feasibility.

3.3.1. Multi-objective optimization with hardware constraints

The proposed system operates under a threat model in which adversaries target IoT and IIoT environments through observable behavioral patterns, including network-based attacks (e.g., denial-of-service, scanning, and malware communication), protocol misuse, and anomalous sensor or process activity. The detection objective is to identify such behaviors at the edge under strict resource constraints using feature representations derived from network traffic, telemetry, and system-level observations. The model assumes that the adversary can influence input data distributions but does not have direct access to the embedded device to modify firmware, extract model parameters, or manipulate the inference process. Consequently, the approach focuses on behavioral intrusion detection rather than physical tampering, cryptographic compromise, or adversarial manipulation of model internals.

The search space $\mathcal{A}$ is defined according to the limitations of Cortex-M microcontrollers and similar platforms. Each candidate architecture is parameterized by its depth (1-3 layers), width (8-128 units), operator type (Conv1D, depthwise separable convolutions, dense layers, and compressed linear operators), and activation function (ReLU6, Tanh, SiLU). Dimensionality-reduction mechanisms, such as global average pooling, are incorporated to limit parameter growth, ensuring that all candidate architectures remain within feasible computational bounds while preserving sufficient diversity for optimization.

Each candidate architecture $\theta \in \mathcal{A}$ is evaluated using a multi-objective cost function that jointly considers detection accuracy and hardware efficiency:

$$\mathcal{J}(\theta) = \mathcal{L}_{\text{det}}(\theta) + \lambda_1 \text{Latency}(\theta) + \lambda_2 \text{Energy}(\theta) + \lambda_3 \text{Memory}(\theta) \quad (4)$$

The detection loss $\mathcal{L}_{\text{det}}(\theta)$ is computed using cross-entropy or focal loss, depending on class imbalance. Latency, energy, and memory are estimated using hardware-aware models calibrated for Cortex-M microcontrollers, incorporating operation counts, execution time, and parameter footprint. Each component of the cost function is normalized to ensure comparability across different scales. The weighting coefficients λ_1 , λ_2 , and λ_3 are defined to balance detection performance and hardware constraints, assigning comparable importance to accuracy, latency, energy consumption, and memory usage under the target deployment profile.

3.3.2. Hybrid NAS optimization procedure

The search process combines differentiable optimization (DARTS) with hardware-aware constraints inspired by ProxylessNAS. Candidate architectures are represented through continuous parameters and optimized using gradient-based updates, enabling efficient exploration of the search space without exhaustive enumeration.

To reduce computational cost, candidate architectures are evaluated using proxy training on subsets of the dataset. Each candidate is assessed based on detection loss and estimated hardware metrics, and poorly performing configurations are progressively discarded.

The search space, defined by combinations of depth, width, operator types, and activation functions, yields several hundred candidate configurations under the imposed TinyML constraints. Architecture parameters are optimized using a gradient-based approach with the Adam optimizer, jointly updating network weights and architecture coefficients during the search phase. Proxy training is conducted using reduced subsets of the dataset over a limited number of epochs, with early stopping based on convergence of validation loss. The search process terminates when no significant improvement in the multi-objective cost function is observed across consecutive iterations, thereby balancing exploration efficiency and computational feasibility. [Algorithm 1](#) describes the complete search procedure.

Algorithm 1 Hardware-aware TinyML neural architecture search (NAS).

```

1: Define search space  $\mathcal{A}$ , dataset  $\mathcal{Z}$ , and hardware profile  $\mathcal{H}$ 
2: Initialize architecture parameters  $\alpha$ 
3: while not converged do
4:   Sample candidate architectures  $\theta_k$ 
5:   for each  $\theta_k$  do
6:     Perform proxy training on  $\mathcal{Z}$ 
7:     Compute detection loss and hardware estimates
8:     Evaluate multi-objective cost  $J(\theta_k)$ 
9:   end for
10:  Update  $\alpha$  via gradient-based optimization
11: end while
12: Select optimal architecture  $\theta^*$ 
13: Fully train  $\theta^*$  using quantization-aware training

```

3.3.3. Final architecture selection and deployment criteria

The final architecture is selected from the Pareto-optimal set of candidates, ensuring a balanced trade-off between accuracy, latency, energy consumption, and memory usage. Only architectures that satisfy strict deployment constraints are considered, including RAM usage below 96 KB, inference latency below 25 ms, and energy consumption below 1 mJ per inference.

The selected model is subsequently trained using quantization-aware training and deployed using TensorFlow Lite Micro. Final validation is performed on the target hardware to ensure consistency between estimated and observed performance. [Fig. 1](#) summarizes the NAS pipeline and the architecture selection process.

3.4. TinyML model training, quantization, and deployment pipeline

The optimal architecture θ^* obtained through NAS is trained using a pipeline designed to ensure stability and compatibility with ultra-low-power microcontrollers. Initial training is performed in a high-performance environment with GPU acceleration, incorporating a warm-up phase and adaptive learning-rate scheduling to stabilize convergence for compact models. This process, illustrated in [Fig. 2](#), produces a baseline model suitable for subsequent compression.

The model is then refined using quantization-aware training (QAT), which simulates 8-bit quantization during optimization. This approach allows the model parameters to adapt to discretization effects, reducing accuracy degradation compared to post-training quantization. Structured pruning is subsequently applied with compression ratios between 90–95%, removing redundant channels and neurons based on sensitivity analysis. This reduces both memory footprint and computational cost while preserving the architecture defined by the NAS process.

After compression, the model is converted for deployment using TensorFlow Lite Micro or microTVM [24]. During this stage, incompatible operators are replaced with optimized kernels targeting SIMD instructions and DSP extensions available on the microcontroller. Model size validation ensures that FLASH usage remains within 192–512 KB and RAM usage below 96 KB. The final model is deployed on an ARM Cortex-M4 microcontroller and evaluated under real-time conditions. Energy consumption is measured using a power analyzer that integrates instantaneous power during inference execution. The observed latency and energy metrics confirm the consistency between the NAS cost model and the actual hardware performance.

3.5. Evaluation protocol and metrics

The system evaluation is designed to assess both detection performance and operational feasibility on embedded hardware, subject to the constraints of Cortex-M-class microcontrollers. This evaluation combines standard classification metrics with direct measurements of latency, energy consumption, memory usage, and temporal stability across heterogeneous domains (TON_IoT, IoT-23, Edge-IIoTset). All hardware-related metrics are obtained through real-world instrumentation, ensuring that the results reflect actual deployment conditions.

Detection performance is evaluated using metrics suitable for imbalanced multiclass intrusion detection scenarios, including Accuracy, Precision, Recall, and F1-score (both micro- and macro-averaged). Among these, the Matthews correlation coefficient (MCC)

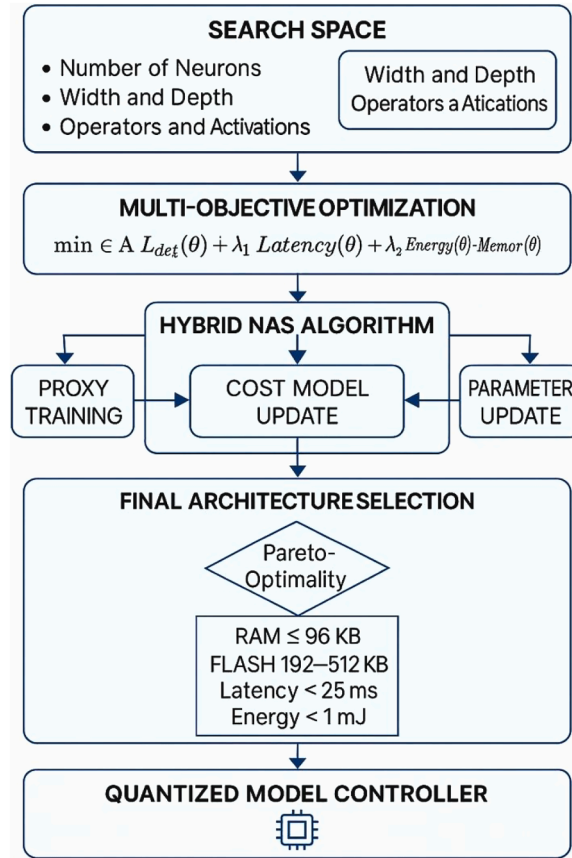


Fig. 1. Hardware-aware NAS pipeline for TinyML intrusion detection.

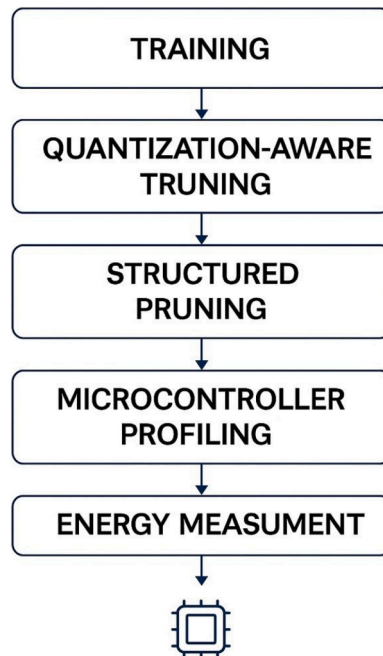


Fig. 2. TinyML training, compression, and deployment workflow.

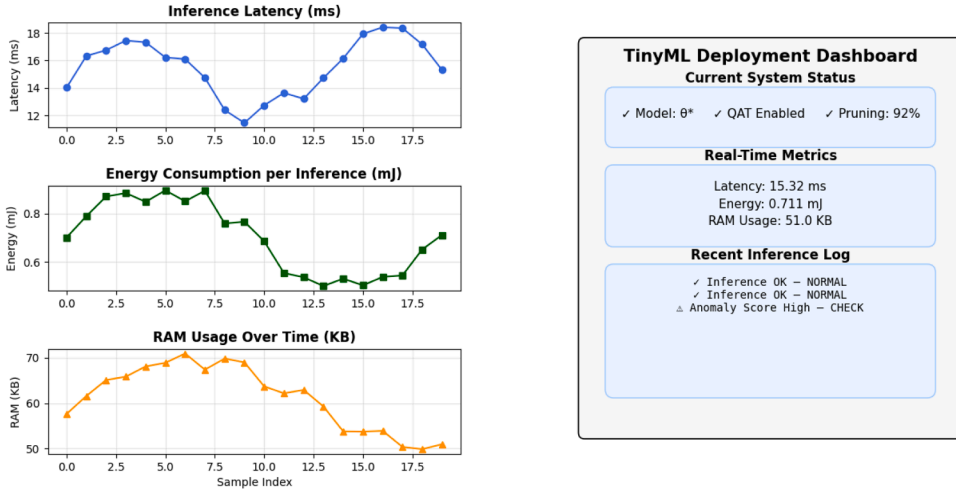


Fig. 3. TinyML deployment dashboard for real-time runtime, energy, and memory evaluation.

is used as the primary robustness metric due to its sensitivity to all elements of the confusion matrix:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (5)$$

Balanced Accuracy and macro-averaged F1-score are additionally considered to ensure stability across minority classes. To further characterize model behavior under high-risk conditions, multi-class ROC and Precision–Recall curves are computed, with particular emphasis on minority attack categories.

3.6. TinyML runtime, resource efficiency, and robustness evaluation

The runtime evaluation assesses the ability of the selected model θ^* to maintain accuracy, computational efficiency, and operational stability under strict hardware constraints. All measurements are performed on an ARM Cortex-M4 microcontroller, ensuring that the reported results reflect real deployment conditions.

RAM usage, including weights, intermediate activations, and input buffers, is recorded alongside the final FLASH footprint after conversion to TensorFlow Lite Micro. Inference latency is measured using embedded timers that capture CPU cycles per sample and convert them to milliseconds. Energy consumption is obtained by integrating current measurements acquired via a precision shunt and a high-resolution ADC, yielding an accurate estimate of per-inference energy cost. The low per-inference energy cost suggests that the proposed model is suitable for continuous or high-frequency monitoring scenarios in resource-constrained environments, without significantly impacting device-level energy budgets.

To evaluate robustness, the model is validated across multiple domains (TON_IoT, IoT-23, Edge-IIoTset), assessing its ability to generalize across different traffic patterns and system behaviors. Temporal disturbances, Gaussian noise, and abrupt signal variations are introduced to simulate partially observable anomalies and short-duration attacks. Sequential drift is also considered to emulate sensor degradation and long-term variability in IoT environments. Stress testing under irregular traffic bursts is performed to verify model stability under high-load conditions.

The experimental configuration includes multilevel cross-validation and direct deployment of the quantized model on the target microcontroller. Comparative evaluation is conducted against conventional models (Random Forest, SVM, compact CNN, and LSTM) and other TinyML configurations.

Fig. 3 presents the real-time monitoring dashboard used during deployment, displaying latency, energy consumption, memory usage, and system-level execution logs. The displayed values correspond to sequential inference operations over a continuous stream of input samples, reflecting real-time execution under non-stationary conditions. Variations in the observed metrics arise from differences in input characteristics and corresponding activation patterns during inference, rather than from measurement noise, providing insight into the model's dynamic behavior under realistic deployment scenarios.

4. Results

4.1. Multidomain characterization of the dataset and feature space alignment

Table 1 summarizes the statistical properties of the four domains, highlighting substantial heterogeneity in sample size, feature dimensionality, and class distribution. IoT-23 and Edge-IIoTset contain more than 2 million samples, whereas TON_IoT and CTD represent intermediate and controlled scenarios, respectively. These differences reflect realistic deployment conditions rather than artificial imbalance.

Table 1
Multidomain dataset statistical characterization.

A	B	C	D	E	F	G	H
TON_IoT	35,975	57	2.240	0.226	0.136	0.807	2.172
IoT-23	2,146,662	8	0.157	0.156	0.109	1.000	0.896
Edge-IIoTset	2,219,201	43	2.677	0.083	0.131	0.907	0.468
CTD (Controlled)	590	11	4.000	0.195	0.137	1.000	1.947

Note: A: Domain, B: Samples, C: Numeric Features, D: Benign / Attack Ratio, E: Mean Value, F: Std. Dev., G: Avg. Dynamic Range, H: Mean Feature Entropy.

Table 2
Dimensional reduction analysis and variance retained by domain.

Domain	Original Features	PCA (90 %)	PCA (95 %)	Retained Variance (32)
TON_IoT	46	14	19	0.999
IoT-23	8	5	6	1.000
Edge-IIoTset	39	20	24	0.998
CTD	11	7	8	1.000

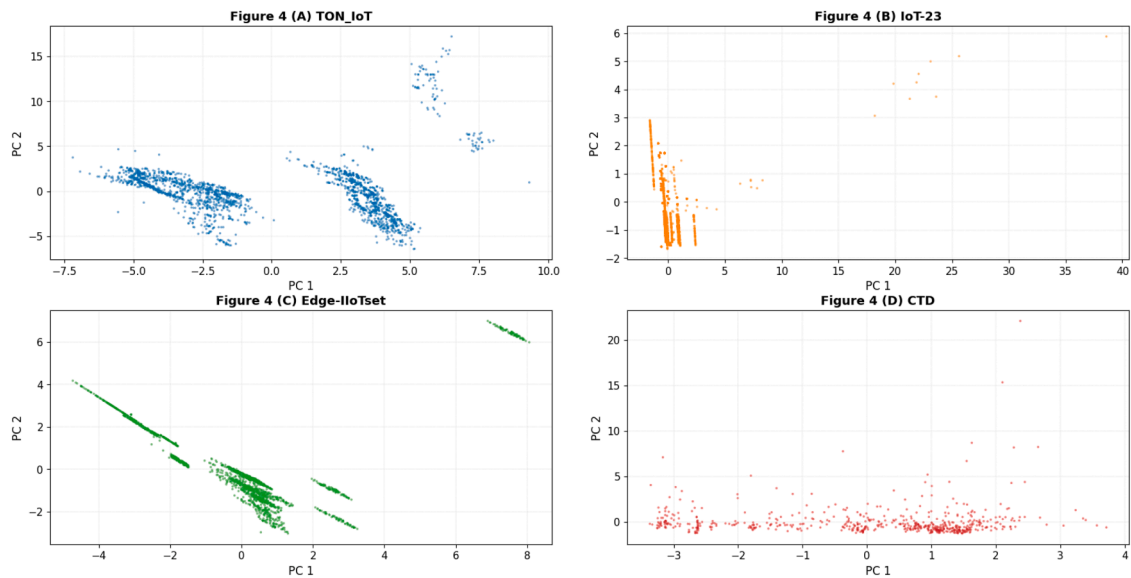


Fig. 4. Two-dimensional projection of the feature space using PCA. (A) corresponds to the TON_IoT dataset, (B) to the IoT-23 dataset, (C) to the Edge-IIoTset dataset, and (D) to the controlled environment dataset (CTD).

From a structural perspective, IoT-23 exhibits a strong dominance of benign traffic, while TON_IoT and Edge-IIoTset show more balanced attack distributions. CTD presents an extreme ratio, consistent with its controlled design. Variability and entropy metrics further indicate that TON_IoT and CTD contain richer feature representations, whereas IoT-23 is more concentrated and homogeneous.

Table 2 evaluates the compressibility of each domain using PCA. Despite differences in initial dimensionality, all datasets admit compact representations, retaining 90-95% of the variance with a limited number of components. Notably, a 32-dimensional representation preserves nearly all informative variance across domains, supporting the feasibility of low-dimensional feature spaces for TinyML deployment.

Fig. 4 provides a geometric characterization of each domain after projection into a common feature space. The observed distributions reveal structural differences that directly impact the feasibility of multidomain modeling. TON_IoT Fig. 4(A) shows intermediate behavior, with correlated clusters associated with multimodal data sources (sensor telemetry, network flows, and logs). This structure suggests recurring patterns but also highlights the complexity introduced by heterogeneous feature types.

IoT-23 Fig. 4(B) exhibits a highly concentrated distribution with limited dispersion, indicating low structural variability dominated by benign traffic. This compact geometry facilitates classification but also reduces the diversity of discriminative patterns, making the domain sensitive to imbalance-aware evaluation.

Edge-IIoTset Fig. 4(C) exhibits a significantly more dispersed structure with multiple dense regions, reflecting greater diversity in operational conditions and traffic behaviors. This increased variability imposes stronger generalization requirements on the model and explains the need for feature alignment before integration.

Table 3
Performance of baseline models in intrusion detection under TinyML constraints.

Domain	Model	F1-macro	F1-micro	Balanced Accuracy	MCC
IoT-23	Random Forest	0.997	0.997	0.997	0.994
	Linear SVM	0.666	0.677	0.677	0.381
	Compact MLP	0.958	0.958	0.958	0.916
CTD	Random Forest	0.982	0.989	0.982	0.964
	Linear SVM	0.499	0.808	0.525	0.155
	Compact MLP	0.502	0.814	0.529	0.215
Combined	Linear SVM	0.789	0.826	0.810	0.585
	Compact MLP	0.972	0.978	0.978	0.945

Table 4
Performance metrics of the NAS-TinyML θ^* model on Cortex-M4 hardware.

Model	Latency (ms)	Dynamic RAM (KB)	FLASH (KB)	Energy per inference (mJ)
Baseline model (without NAS)	5.87	34.6	118.3	1.21
Quantized model	3.92	26.1	71.8	0.73
θ^* (NAS-TinyML)	2.48	18.7	52.4	0.39

The CTD dataset Fig. 4(D) has a fragmented distribution with well-separated clusters, consistent with its controlled design. While its scale is limited, the internal consistency of these clusters supports its role as a complementary domain for evaluating robustness under constrained conditions. These structural differences explain why direct cross-domain training is not feasible without explicit feature-space alignment and justify the multidomain integration strategy adopted in this work.

4.2. Detection performance of the selected model using NAS (θ^*)

Table 3 summarizes the detection performance of baseline models across the evaluated domains. Random Forest is included as a high-capacity reference, while the linear SVM and compact MLP represent models compatible with embedded deployment.

In IoT-23, the models operate under favorable conditions due to the compact and well-structured feature space. The Random Forest achieves near-perfect performance, establishing an upper bound, while the compact MLP reaches competitive results, indicating that low-capacity neural architectures can capture relevant nonlinear patterns. The linear SVM shows a significant degradation, confirming the nonlinearity of the decision boundaries in this domain.

The CTD domain presents a more challenging scenario due to its controlled imbalance. While the Random Forest maintains high performance, both the SVM and compact MLP exhibit reduced macro-averaged metrics, reflecting sensitivity to class imbalance. The divergence between micro- and macro-based metrics highlights the importance of using imbalance-aware evaluation criteria.

In the multidomain scenario, where heterogeneous datasets are integrated into a shared feature space, the compact MLP achieves strong performance, demonstrating effective generalization across aligned domains. The SVM shows reduced performance, indicating that linear decision boundaries are insufficient to capture cross-domain patterns.

The TON_IoT and Edge-IIoTset domains are excluded from supervised evaluation after preprocessing, as their final distributions collapse into a single effective class. Under these conditions, discriminative learning is not statistically well-defined, and any reported metrics would be artificially biased. This behavior reflects structural properties of the data rather than limitations of the model.

Fig. 5 illustrates the structural factors that determine the feasibility of cross-domain evaluation. The class distribution after preprocessing Fig. 5(A) shows that only IoT-23 and CTD retain sufficient diversity for supervised learning, while TON_IoT and Edge-IIoTset collapse into a single effective class.

The feature-space comparison Fig. 5(B) reveals substantial differences in dimensionality across domains, preventing direct transferability without alignment. This limitation is further confirmed by the feature intersection analysis (Fig. 5C), which shows minimal overlap between domains, indicating the absence of a shared input space suitable for cross-domain training.

The feasibility map Fig. 5(D) summarizes these constraints, showing that only IoT-23 and CTD satisfy the combined requirements of class diversity and feature compatibility. These conditions define the valid scope for supervised cross-domain evaluation and explain the exclusion of structurally degenerate domains.

4.3. Runtime performance on microcontroller hardware (latency, RAM, energy)

Table 4 reports the hardware-level performance of the evaluated models on a Cortex-M4 microcontroller, including inference latency, dynamic RAM usage, FLASH footprint, and energy consumption per inference.

The results show a consistent improvement across all metrics as optimization stages are applied. While quantization provides an initial reduction in latency and memory usage, the θ^* model achieves the most significant gains, reducing inference time from 5.87 ms to 2.48 ms. This reduction indicates that structural optimization introduced by NAS contributes more substantially to execution efficiency than quantization alone.

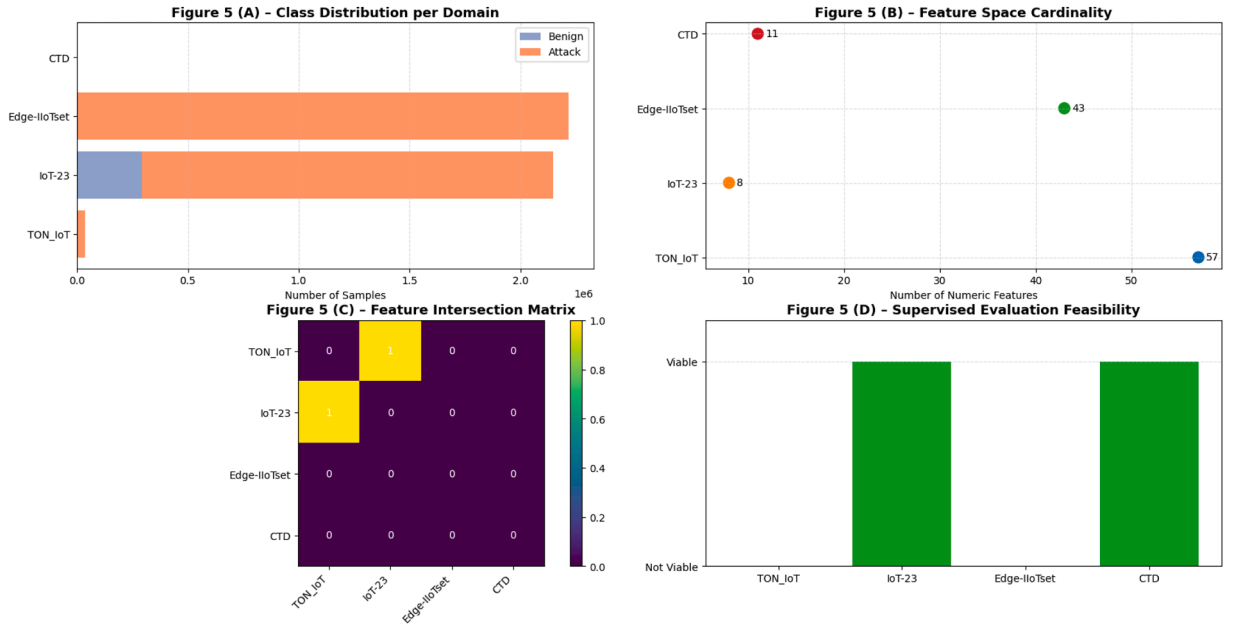


Fig. 5. Structural conditions for cross-domain evaluation in intrusion detection; (A) class distribution after preprocessing in each domain; (B) adequate overlap of numerical features between domains; (C) feasibility of supervised training under PCA alignment; (D) domains excluded by class degeneracy or structural incompatibility.

Table 5

Relative performance degradation under robustness stress tests across domains ($\Delta F1$ -macro, %).

A) Noise robustness (Gaussian noise on normalized features)				
Domain	$\sigma = 0.03$	$\sigma = 0.06$	$\sigma = 0.10$	$\sigma = 0.15$
IoT-23	-0.84	-1.91	-3.72	-6.14
CTD	-1.63	-3.88	-7.46	-12.39
Combined	-1.12	-2.57	-5.21	-8.67
B) Temporal drift stability (sequential windows)				
Domain	W2	W3	W4	
IoT-23	-1.08	-2.36	-3.95	
CTD	-2.71	-5.64	-9.18	
Combined	-1.79	-3.92	-6.83	

A similar trend is observed in memory usage. Dynamic RAM is reduced from 34.6 KB in the baseline model to 18.7 KB in the θ^* configuration, while the FLASH footprint decreases from 118.3 KB to 52.4 KB. These reductions reflect not only parameter compression but also a simplified computational graph with fewer intermediate activations.

Energy consumption follows the same pattern, decreasing from 1.21 mJ to 0.39 mJ per inference. This behavior is directly associated with reductions in both latency and computational load, highlighting the importance of architecture-level optimization for energy-efficient deployment. These results indicate that structural optimization introduced by NAS is a primary factor in improving hardware efficiency, particularly under strict memory and energy constraints typical of low-power microcontrollers.

4.4. Robustness and cross-Domain generalization analysis

Table 5 reports the relative performance degradation under noise injection and temporal drift conditions. The results show a progressive degradation pattern across all domains, without abrupt collapses. IoT-23 exhibits the highest robustness, with only limited degradation under moderate noise levels, consistent with its compact and homogeneous feature space. CTD shows greater sensitivity to perturbations, reflecting its lower data volume and stronger dependence on specific signal patterns. The combined domain exhibits intermediate behavior, indicating that structural diversity helps mitigate the impact of localized disturbances.

A similar pattern is observed under temporal drift. IoT-23 maintains relatively stable performance across sequential windows, whereas CTD exhibits a more pronounced degradation, suggesting higher sensitivity to non-stationary conditions. The combined domain again demonstrates moderated degradation, indicating that multidomain integration improves temporal stability. All robustness evaluations are performed on the final optimized model θ^* after quantization-aware training and structured pruning, ensuring that the reported behavior reflects the impact of aggressive model compression under realistic deployment conditions.

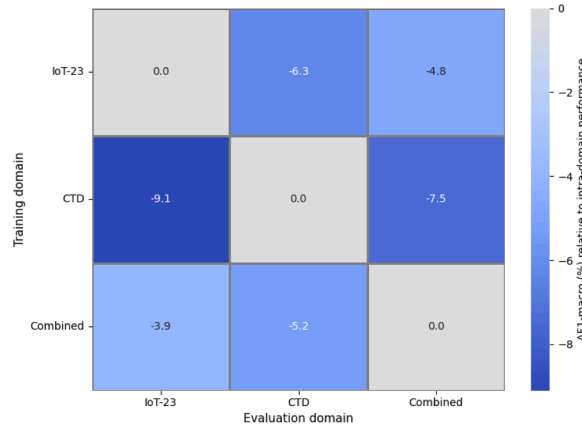


Fig. 6. Relative cross-domain performance degradation of the model under domain shift ($\Delta F1$ -macro, %).

Table 6

Ablation study of architectural and optimization components.

A	B	C	D	E	F	G	H	I
θ^* (final)	Yes	Yes	Yes	Yes	Yes	0.972	2.48	18.7
– QAT	Yes	No	Yes	Yes	Yes	0.975	3.41	19.2
– Pruning	Yes	Yes	No	Yes	Yes	0.978	3.12	27.9
– HW constraints	Yes	Yes	Yes	No	Yes	0.981	4.86	34.1
– Hybrid dataset	Yes	Yes	Yes	Yes	No	0.943	2.44	18.5
Manual baseline (MLP)	No	No	No	No	Yes	0.931	5.87	34.6

Note: A: Configuration, B: NAS, C: QAT, D: Pruning, E: HW constraints, F: Hybrid dataset, G: F1-macro, H: Latency (ms), I: RAM (KB).

Fig. 6 illustrates cross-domain performance degradation under domain shift. The results reveal asymmetric transfer behavior, where models trained on CTD exhibit larger performance drops when evaluated on IoT-23, indicating limited generalization from controlled environments to more complex traffic conditions. In contrast, training on IoT-23 leads to smaller degradation when evaluated on CTD, suggesting that representations learned from diverse traffic patterns are more transferable.

When the combined domain is used for training, degradation remains moderate across target domains, indicating that multidomain integration produces decision boundaries less dependent on a single statistical structure. However, when used as the evaluation domain, performance depends on the training source, with stronger degradation observed for models trained on CTD, reflecting its narrower distribution.

4.5. Ablation studies and NAS search benefits

CNN and compact LSTM architectures were evaluated as exploratory baselines; however, they are not included in the main comparative analysis because they cannot meet the strict memory and latency constraints required for deployment on TinyML platforms. Table 6 presents the ablation analysis of the selected architecture θ^* , quantifying the impact of each optimization component on both predictive performance and hardware efficiency. The manual MLP baseline exhibits the highest latency and memory usage, indicating that conventional design approaches do not efficiently explore the constraints imposed by embedded environments. In contrast, the optimized configurations progressively reduce computational cost while maintaining competitive performance.

Removing QAT leads to a slight increase in F1-macro. Still, it increases latency and memory consumption, indicating that QAT primarily contributes to numerical stability and efficient low-precision deployment rather than maximizing floating-point accuracy.

A similar trade-off is observed when pruning is removed. Although classification performance improves slightly, resource usage increases significantly, highlighting the role of structural compression in controlling execution costs. When hardware constraints are excluded during NAS, the resulting architecture achieves competitive accuracy but exceeds feasible limits in latency and memory usage. This behavior indicates that constraint-aware optimization must be incorporated into the search process rather than applied as a post hoc adjustment.

Eliminating the hybrid dataset reduces model generalization, reflected in a drop in F1-macro while maintaining similar hardware efficiency. This pattern indicates that multidomain training directly contributes to robustness without increasing computational cost. The final model θ^* operates in a region where accuracy, latency, and memory usage are jointly balanced, avoiding configurations that optimize a single metric at the expense of deployment feasibility.

Fig. 7 illustrates this trade-off through the Pareto front obtained during NAS. Early architectures achieve high accuracy but low efficiency, while successive iterations converge toward compact configurations with reduced latency and memory usage. The

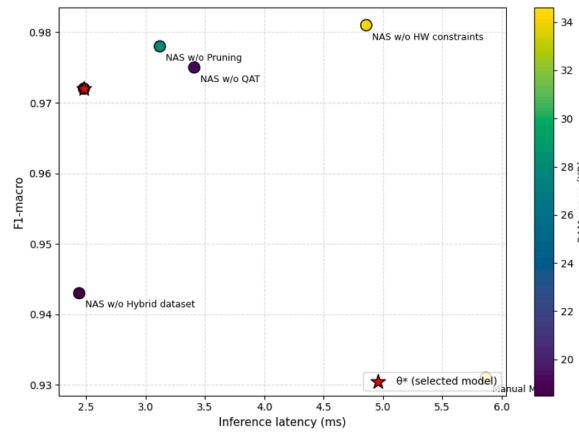


Fig. 7. Pareto; trade-off between detection performance and computational efficiency under TinyML constraints.

position of θ^* reflects a non-dominated solution within this space, where further efficiency gains would require disproportionate losses in predictive performance.

5. Discussion

The findings align with prior research in IoT intrusion detection, where high-capacity models trained in single domains tend to overfit and exhibit limited transferability under heterogeneous conditions [25,26]. In contrast, models with controlled nonlinear capacity provide a more stable balance between accuracy and generalization [27]. The present study extends this perspective by incorporating hardware constraints as a primary design variable, linking predictive performance with deployment feasibility in TinyML environments.

The multidomain alignment and feature-reduction process plays a central role in achieving this balance. The results indicate that domains with substantial differences in scale, distribution, and feature semantics can be projected into a compact latent space without compromising discriminative capacity. This behavior supports the feasibility of unified pipelines for heterogeneous IoT data, provided that explicit normalization and feature selection mechanisms are applied.

The comparative evaluation of baseline models highlights the dependency of performance on structural properties of the data. Favorable domains with balanced distributions support high accuracy even for simple models, whereas controlled or low-volume domains introduce instability, particularly in compact architectures. This sensitivity emphasizes the need for robust evaluation metrics and careful dataset characterization when designing embedded detection systems.

A critical aspect of this study is the identification of structural limitations in cross-domain evaluation. The presence of degenerate domains after preprocessing indicates that not all datasets support valid supervised training under realistic conditions. This observation challenges the common assumption that cross-domain generalization can be assessed uniformly across benchmarks and highlights the need for prior feasibility analysis before model training.

Robustness experiments under noise and temporal drift conditions show that performance degradation remains gradual, suggesting that the model captures stable patterns rather than domain-specific artifacts. The variability observed across domains indicates that resilience is directly influenced by the data's statistical structure, reinforcing the value of multidomain training strategies.

Several limitations should be considered. Cross-domain evaluation is constrained by the availability of valid class distributions, limiting the scope of generalization analysis. Although hardware performance is validated on Cortex-M platforms, variations may arise across different microcontroller architectures. Additionally, the approach relies on predefined numerical features, which may not fully capture evolving traffic patterns in highly dynamic environments.

The contribution of this work lies in the joint integration of feature-space alignment across heterogeneous datasets, hardware-aware neural architecture search under TinyML constraints, and validation on real microcontroller hardware. This combination enables the development of a practical intrusion detection framework that operates within strict resource limitations while maintaining stable performance across multiple domains. The proposed system is designed as a device-level intrusion detection component under TinyML constraints. It can be integrated as a building block within larger distributed and multi-layer IoT security architectures.

6. Conclusions

This study addresses intrusion detection in IoT environments under TinyML constraints by jointly integrating multidomain feature-space alignment, hardware-aware neural architecture search, and validation on real microcontroller hardware. The results show that detection performance in embedded systems is strongly influenced by the structural properties of the data and its compatibility with constrained execution environments.

The proposed approach demonstrates that compact feature representations and constraint-aware architecture design enable efficient deployment while maintaining stable detection performance across heterogeneous domains. By incorporating hardware constraints directly into the NAS process, the resulting model achieves a consistent balance between predictive capability and resource efficiency, supporting practical implementation on Cortex-M class devices. Furthermore, the analysis of cross-domain behavior indicates that differences in dataset structure directly affect model transferability, highlighting the importance of controlled alignment and validation before deployment. Robustness experiments show progressive performance degradation under noise and temporal drift, without abrupt instability, indicating that the optimized model preserves relevant discriminative patterns under perturbation.

This study presents several limitations that should be considered. First, the evaluation is restricted to a set of publicly available datasets and a controlled experimental environment, which, despite covering heterogeneous scenarios, may not fully represent the variability of real-world IoT deployments. Second, while multidomain alignment improves generalization, residual domain-shift effects persist, particularly when transferring from controlled environments to more complex traffic conditions. Third, the analysis focuses on model-level robustness and hardware efficiency, without explicitly addressing class-specific calibration or fine-grained adversarial resilience. Future work will focus on adaptive and federated learning strategies under TinyML constraints, broader validation across additional hardware platforms, and more comprehensive evaluation under diverse operational conditions.

CRedit authorship contribution statement

Iván Ortiz-Garcés: Validation, Software, Methodology, Data curation; **Milton Román-Cañizares:** Visualization, Software, Methodology, Formal analysis, Conceptualization; **Pablo Palacios:** Writing – review & editing, Validation, Software, Investigation, Conceptualization; **William Villegas-Ch:** Writing – review & editing, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] M.N.A. Ramadan, M.A.H. Ali, S.Y. Khoo, M. Alkhedher, Federated learning and TinyML on IoT edge devices: challenges, advances, and future directions, *ICT Express* 11 (4) (2025) 754–768. <https://doi.org/10.1016/j.icte.2025.06.008>
- [2] C. Yahyati, I. Lamaakal, Y. Maleh, K. El Makkaoui, I. Ouahbi, M. Almousa, A.A. Abd El-Latif, A systematic review of state-of-the-art TinyML applications in healthcare, education, and transportation, *IEEE Access* (2025). <https://doi.org/10.1109/ACCESS.2025.3633575>
- [3] A. Alfahaid, E. Alalwany, A.M. Almars, F. Alharbi, E. Atlam, I. Mahgoub, Machine learning-based security solutions for IoT networks: a comprehensive survey, *Sensors* 25 (11) (2025). <https://doi.org/10.3390/s25113341>
- [4] N. Schizas, A. Karras, C. Karras, S. Sioutas, TinyML for ultra-low power AI and large scale IoT deployments: a systematic review, *Fut. Internet* 14 (12) (2022). <https://doi.org/10.3390/fi14120363>
- [5] S.E. Haddouti, W. Lazraq, TinyML strategies for privacy-preserving and cyber threat multi-classification in edge-IoT networks, *Computing* 107 (8) (2025). <https://doi.org/10.1007/s00607-025-01522-y>
- [6] N. Nedungadi, S. Sankaran, K. Achuthan, Towards a lightweight hybrid multimodal approach for intrusion detection in edge-enabled IoT devices, *Clust. Comput.* 28 (16) (2025) 1010. <https://doi.org/10.1007/s10586-025-05723-0>
- [7] L. Dutta, S. Bharali, TinyML meets IoT: a comprehensive survey, *Internet Things* 16 (2021). <https://doi.org/10.1016/j.iot.2021.100461>
- [8] J.D. Velasquez, L. Cadavid, C.J. Franco, Emerging trends and strategic opportunities in tiny machine learning: a comprehensive thematic analysis, *Neurocomputing* 648 (2025). <https://doi.org/10.1016/j.neucom.2025.130746>
- [9] H. Sebestyen, D.E. Popescu, R.D. Zmaranda, A literature review on security in the Internet of Things: identifying and analysing critical categories, *Computers* 14 (2) (2025). <https://doi.org/10.3390/computers14020061>
- [10] R. Selvaraj, V.M. Kuthadi, S. Baskar, R. Acevedo, Tiny ML-enabled energy-efficient intrusion detection system for sustainable IoT security in green cybersecurity ecosystems, *J. Internet Serv. Inf. Secur.* 15 (3) (2025) 602–625. <https://doi.org/10.58346/JISIS.2025.13.041>
- [11] C. Amuthadevi, R. Venkatesan, M. Mythily, R.A. Canessane, TinyML-based intrusion detection systems for sustainable and energy-constrained IoT devices, *Results Eng.* 28 (2025). <https://doi.org/10.1016/j.rineng.2025.108013>
- [12] A. Hamdouchi, A. Idri, Evaluating the performance of TinyML singular and ensemble techniques for intrusion detection in IoT networks, *Microprocess. Microsyst.* 117 (2025). <https://doi.org/10.1016/j.micpro.2025.105172>
- [13] A. Sharma, S. Rani, M. Shabaz, An optimized stacking-based TinyML model for attack detection in IoT networks, *PLOS ONE* 20 (8) (2025). <https://doi.org/10.1371/journal.pone.0329227>
- [14] Z. Alwaisi, Memory-efficient and robust detection of Mirai botnet for future 6G-enabled IoT networks, *Internet Things* 32 (2025). <https://doi.org/10.1016/j.iot.2025.101621>
- [15] V. Tsoukas, A. Gkogkidis, A. Kakarountas, Elements of TinyML on constrained resource hardware, in: *Advances in Computing and Data Sciences*, Springer International Publishing, Cham, 2022, pp. 316–331.
- [16] A. Porsia, A. Ruospo, E. Sanchez, Model theft attack against a TinyML application running on an ultra-low-power open-source SoC, in: *Proceedings of the 21st ACM International Conference on Computing Frontiers Companion*, 2024, pp. 63–66. <https://doi.org/10.1145/3637543.3652877>
- [17] T. Magara, Y. Zhou, Internet of Things (IoT) of smart homes: privacy and security, *J. Electr. Comput. Eng.* 2024 (2024). <https://doi.org/10.1155/2024/7716956>
- [18] T.M. Booi, I. Chiscop, E. Meeuwissen, N. Moustafa, F.T.H. den Hartog, ToN-IoT: the role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets, *IEEE Internet Things J.* 9 (1) (2022) 485–496. <https://doi.org/10.1109/JIOT.2021.3085194>
- [19] S. Garcia, A. Parmisano, M.J. Erquiaga, IoT-23: a labeled dataset with malicious and benign IoT network traffic, Zenodo, 2020, <https://doi.org/10.5281/zenodo.3749788>

- [20] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning, *IEEE Access* 10 (2022). <https://doi.org/10.1109/ACCESS.2022.3165809>
- [21] Y.A. Soliman, A.S. Ghoneim, M.M. Elkhoully, A comprehensive systematic review of TinyML for person detection systems, *IAENG Int. J. Comput. Sci.* 52 (11) (2025) 4074–4086.
- [22] E.A. Asonye, S.M. Musa, C.M. Akujuobi, M.N.O. Sadiku, J. Foreman, Realizing an IoT-based home area network model using ZigBee in the global environment, *Int. J. Comput. Digit. Syst.* 9 (6) (2020) 1011–1020. <https://doi.org/10.12785/ijcds/0906011>
- [23] W. Song, X. Zhu, S. Ren, W. Tan, Y. Peng, A hybrid blockchain and machine learning approach for intrusion detection system in industrial Internet of Things, *Alex. Eng. J.* 127 (2025) 619–627. <https://doi.org/10.1016/j.aej.2025.05.030>
- [24] D.J.P. Manajang, S.R. U.A. Sompie, A. Jacobus, Implementasi framework TensorFlow object detection API dalam mengklasifikasi jenis kendaraan bermotor, *J. Tek. Inform.* 15 (3) (2020).
- [25] V.-E. Baci, A. Braeken, L. Segers, B. da Silva, Secure tiny machine learning on edge devices: a lightweight dual attestation mechanism for machine learning, *Fut. Internet* 17 (2) (2025). <https://doi.org/10.3390/fi17020085>
- [26] P. Fusco, G.P. Rimoli, M. Ficco, TinyIDS: an IoT intrusion detection system by tiny machine learning, in: *Lecture Notes in Computer Science*, 14816, Springer, 2024, pp. 71–82. https://doi.org/10.1007/978-3-031-65223-3_5
- [27] M. Tawfik, A.H. Abdelhaliem, I.S. Fathi, Transforming IoT security through large language models: a comprehensive systematic review and future directions, *Stat. Optim. Inf. Comput.* 14 (2) (2025) 1018–1044. <https://doi.org/10.19139/soic-2310-5070-2424>